

Định vị: Xây dựng giải pháp phần mềm hướng kiến trúc doanh nghiệp.

Định vị: Xây dựng giải pháp phần mềm hướng kiến trúc doanh nghiệp.

**Lập trình
Ứng dụng Web
Doanh nghiệp
(ECO2415)**

Công nghệ Trục:
C# / .NET & Blazor
Fullstack (Đồng nhất
UI đến CSDL).

Phương pháp: Dự án Thực chiến
(Project-Based Learning) bởi Bộ môn
Tin học Kinh tế - Khoa HTTKT.

Thị trường Web: Phân khúc Startup vs. Doanh nghiệp



Phân khúc Startup & B2C Web

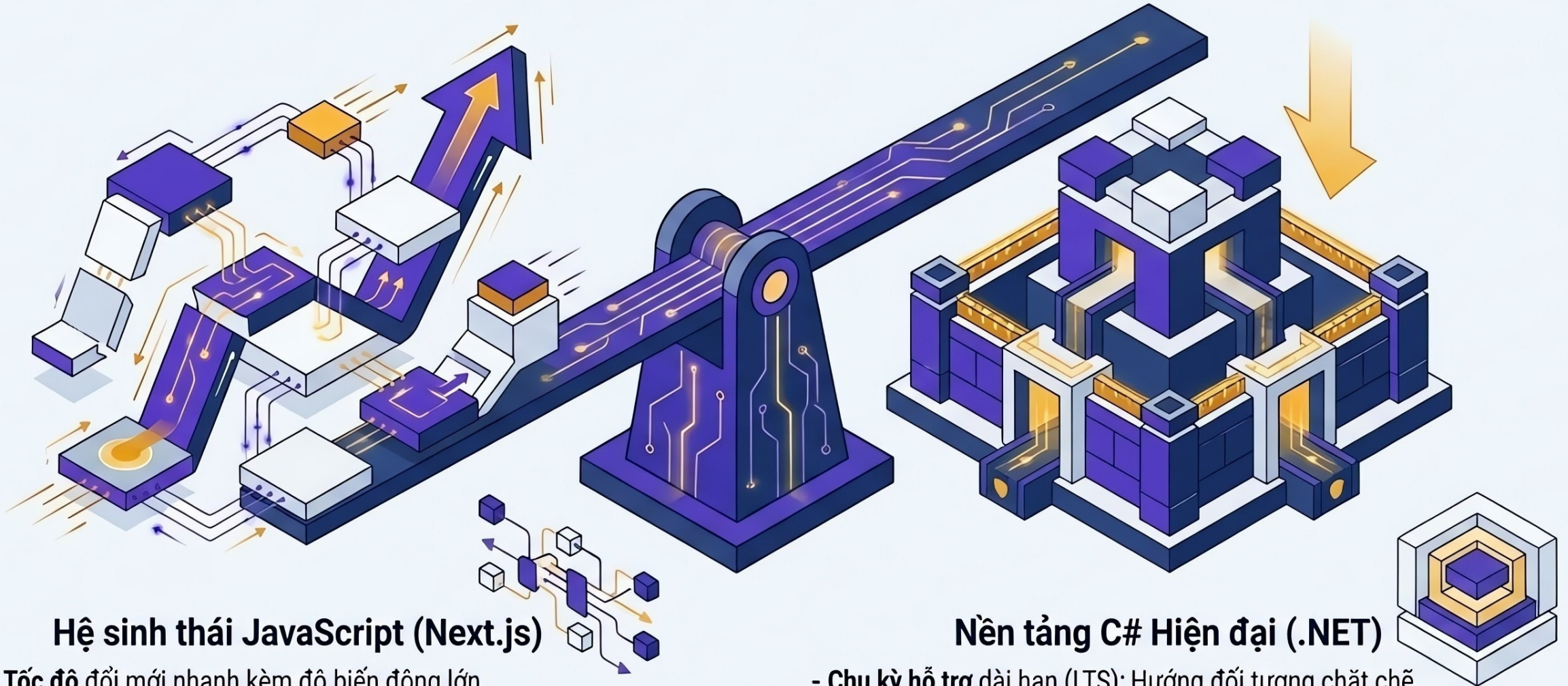
- **Ưu tiên:** Tốc độ ra mắt (Time-to-market), Tối ưu SEO.
- **Công nghệ:** Next.js, React, Node.js.



Phân khúc Enterprise & Tài chính - Ngân hàng

- **Ưu tiên:** An toàn kiểu (Type Safety), Bảo mật, Ổn định dài hạn.
- **Công nghệ:** .NET / C#, Java Spring Boot, RDBMS.

Giải mã Xu hướng: .NET/C# hay Next.js trong Kỷ nguyên AI?



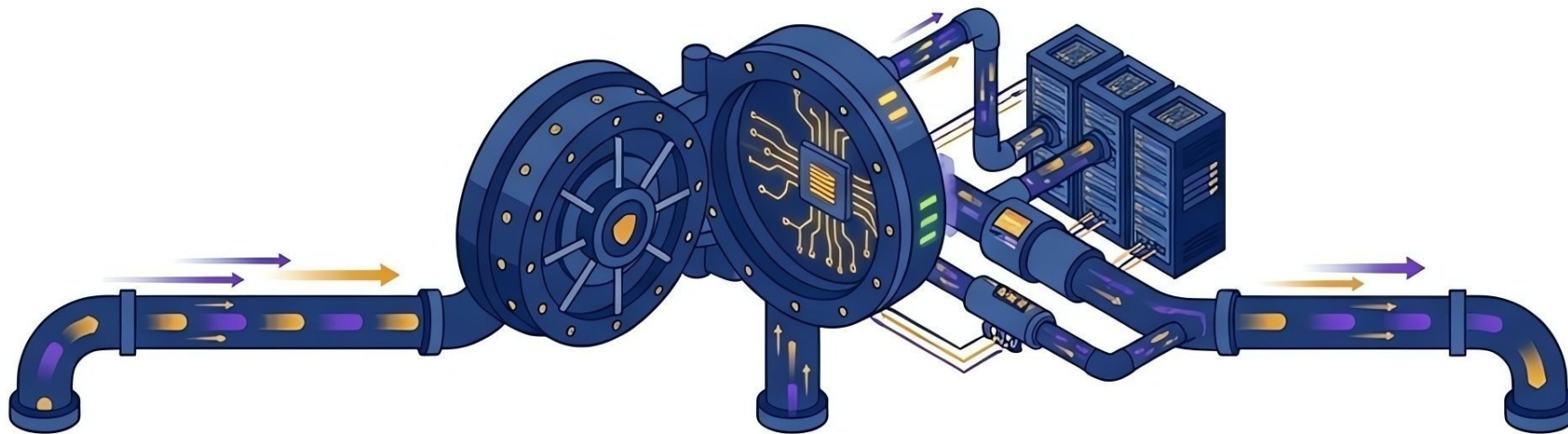
Hệ sinh thái JavaScript (Next.js)

- **Tốc độ** đổi mới nhanh kèm độ biến động lớn.
- **Rủi ro:** Dễ rơi vào bẫy "Spaghetti Code" (trộn lẫn UI và DB).

Nền tảng C# Hiện đại (.NET)

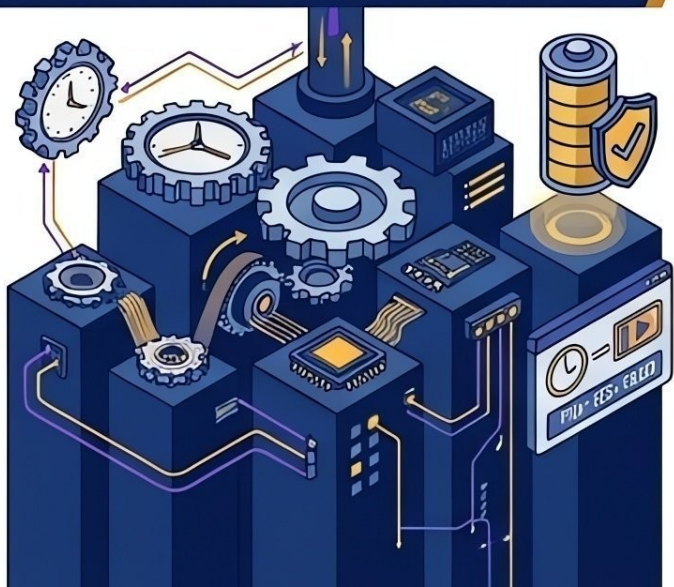
- **Chu kỳ hỗ trợ** dài hạn (LTS); Hướng đối tượng chặt chẽ.
- **Lợi thế:** Rèn luyện tư duy Kiến trúc Sạch (Clean Architecture) bền vững.

Vì sao Ngân hàng & Doanh nghiệp Lớn Trung thành với .NET?



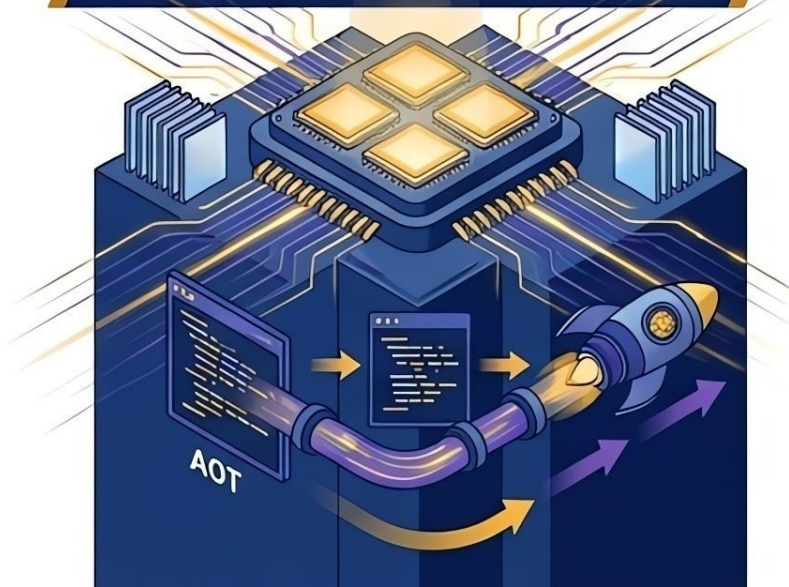
Độ tin cậy

Vận hành bền bỉ 10-15 năm,
tính tương thích ngược tuyệt đối.



Hiệu năng (High Throughput)

Xử lý giao dịch chịu tải cực cao
qua biên dịch Native AOT.



Nghề nghiệp

Nhu cầu tuyển dụng kỹ sư .NET luôn
nằm trong top đầu tại thị trường Việt Nam.



Hệ sinh thái AI Cấp Doanh nghiệp của Microsoft



Blazor Smart Components

Linh kiện UI tự động hóa nhập liệu và tìm kiếm ngữ nghĩa.



Microsoft Semantic Kernel

Mega workflow orchestration engine.
Khung điều phối AI Agents & quy trình RAG.



Microsoft.Extensions.AI

Chuẩn hóa giao diện kết nối OpenAI, Azure, Ollama qua cấu hình DI.

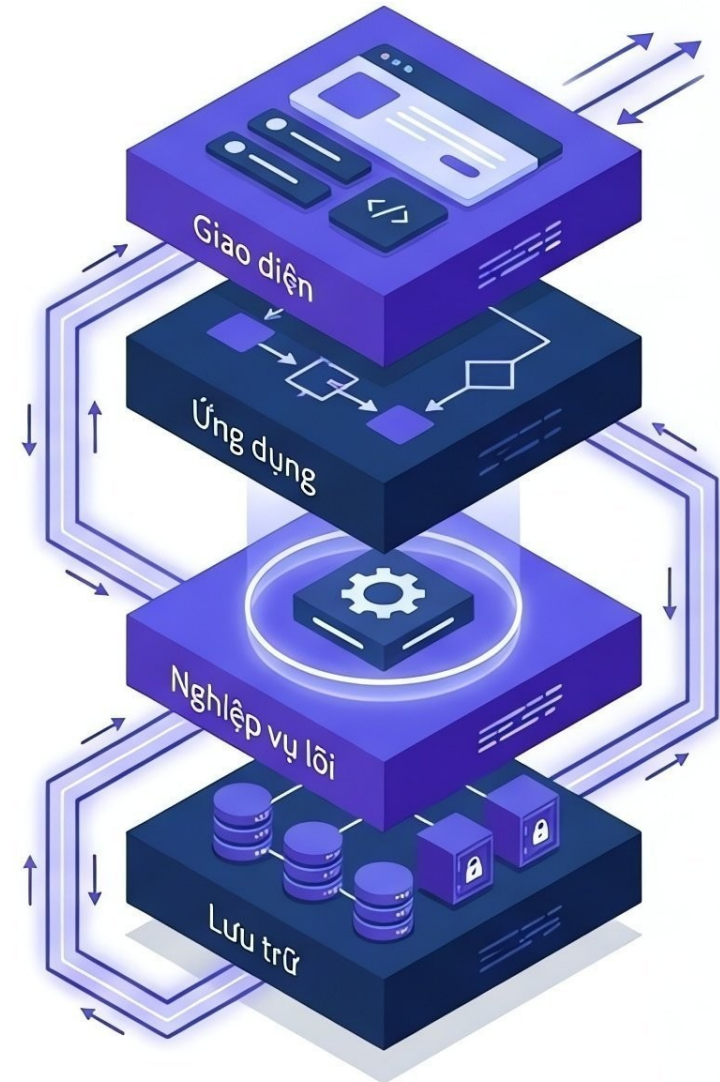


Triết lý Học phần: Code Mì ăn liền vs Kiến trúc Sạch



Vấn nạn "Spaghetti Code"

- Trộn lẫn Giao diện (UI) + Logic + Truy vấn CSDL.
- Hệ quả: Sụp đổ khi mở rộng, không thể bảo trì và kiểm thử.



Giải pháp: Clean Architecture

- Tách biệt rõ ràng 4 tầng trách nhiệm.
- Quy tắc: Nghiệp vụ lõi hoàn toàn độc lập với công nghệ bên ngoài.

Phương pháp Học tập: 35 Video Hands-on

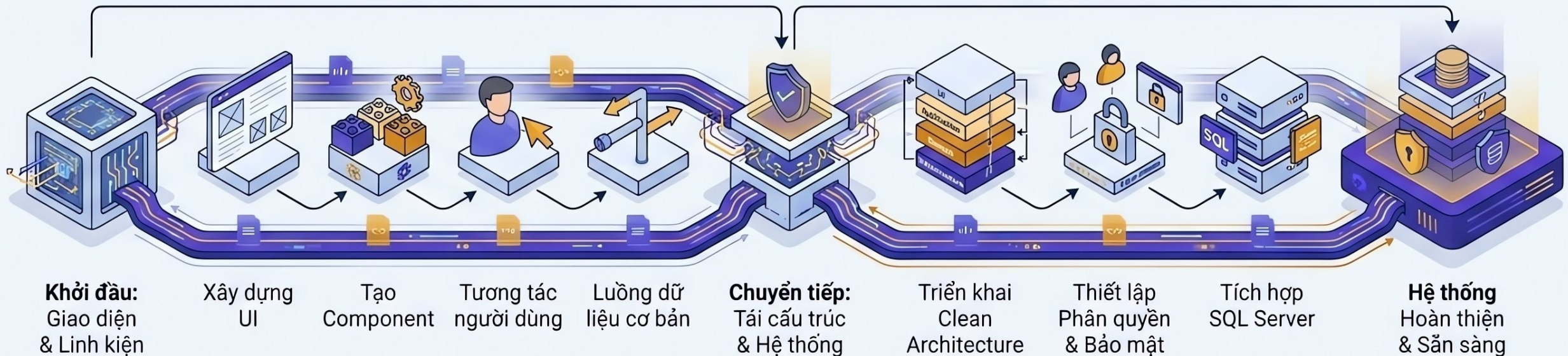
Lớp học Đảo ngược (Flipped Classroom)

- Giảng viên định hướng tư duy tại lớp (Anchor Lectures).
- Sinh viên chủ động thực hành qua video.

Dự án Trục eShop

Giai đoạn 1: Xây dựng giao diện cơ bản & Linh kiện tương tác (Part 1-2).

Giai đoạn 2: Tái cấu trúc Clean Architecture, Phân quyền & SQL (Part 3-5).



Khám phá Dự án Trục: Blazor eShop Enterprise

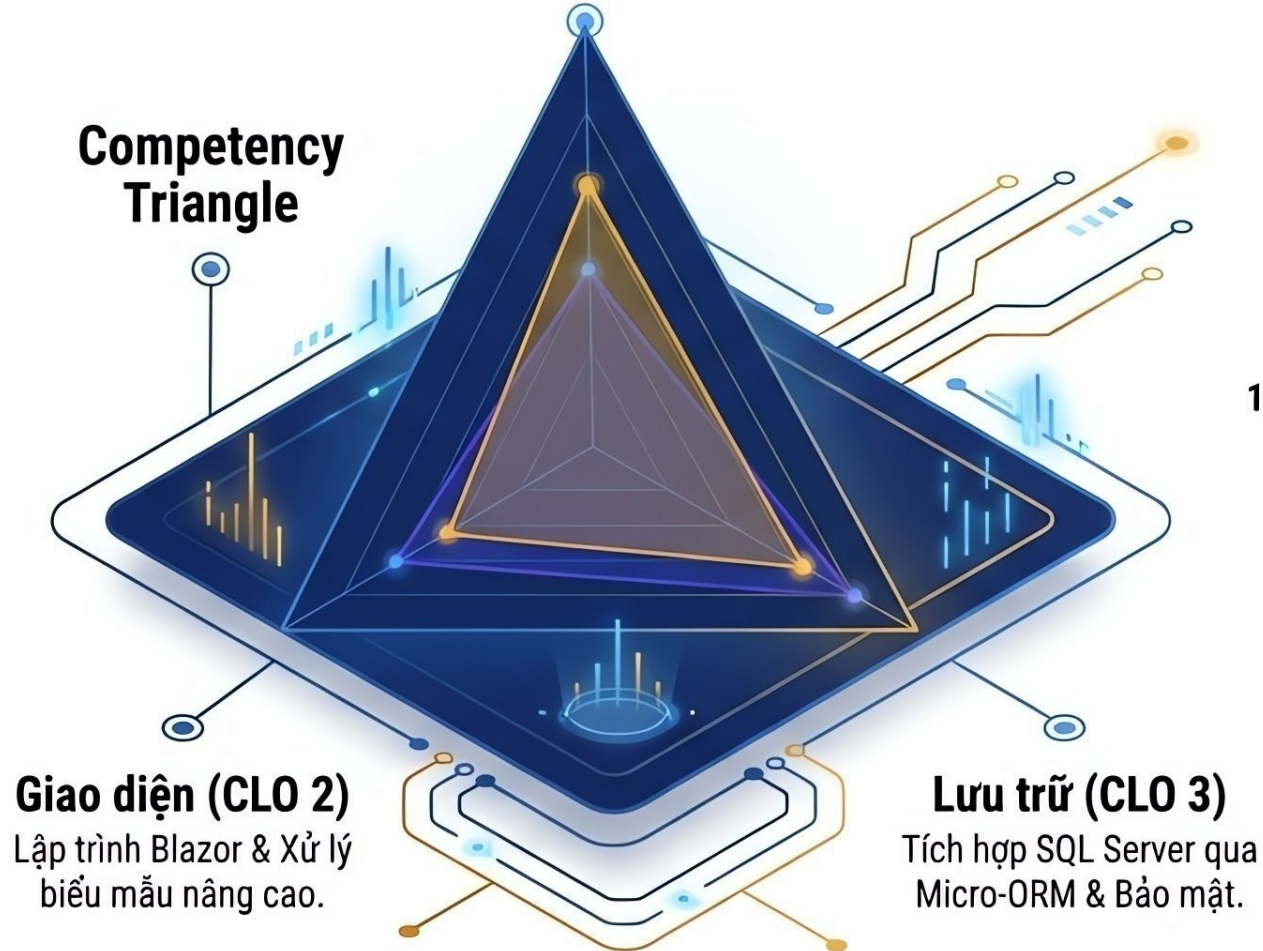


Chuẩn Đầu ra (CLOs) & Trọng số Đánh giá

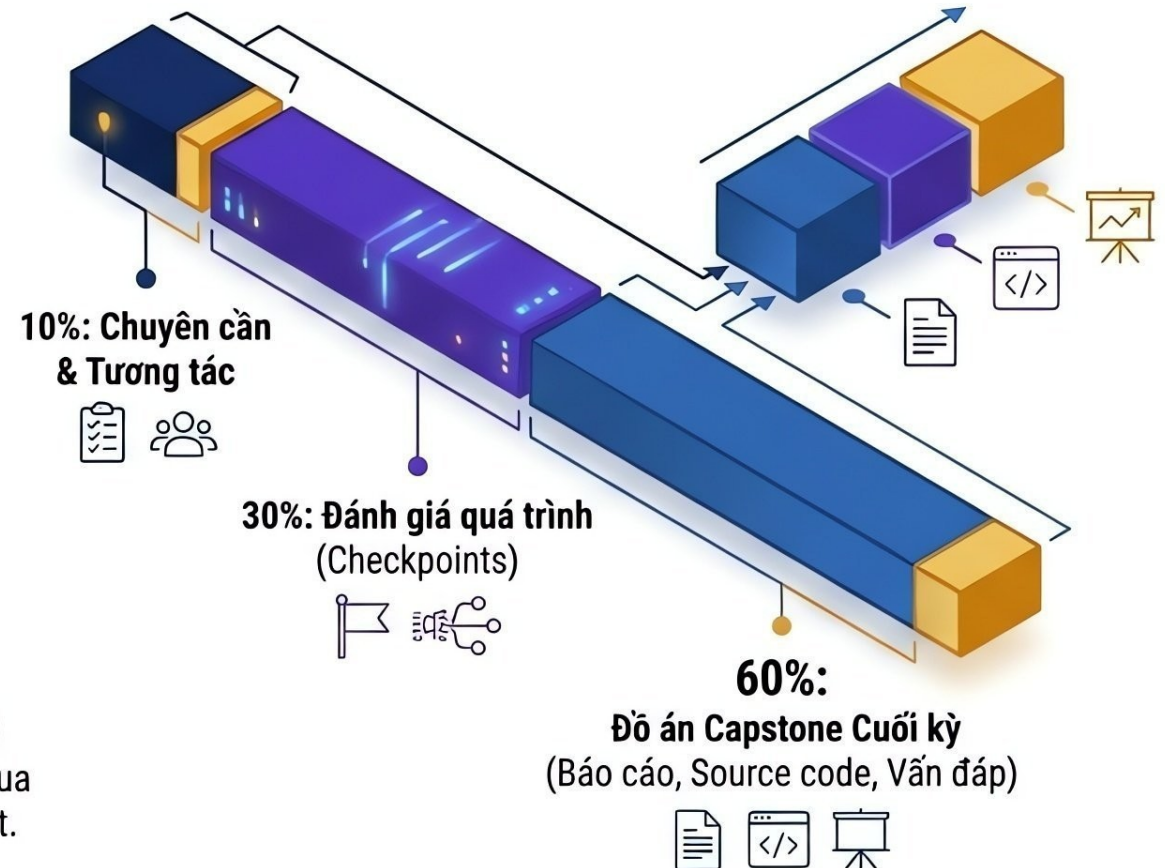
Thiết kế (CLO 1)

Phân tích UML & Clean Architecture.

Competency Triangle



Cơ cấu Đánh giá (10-30-60)



Văn hóa Kỹ sư: Clean Code & Quy chuẩn Đặt tên

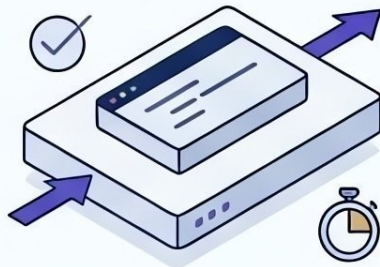


Chuẩn Đặt tên Microsoft C#

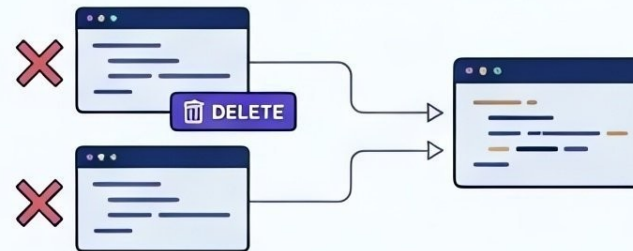
- **PascalCase**: Tên Lớp, Hàm, Thuộc tính (ProductRepository).
- **camelCase**: Biến cục bộ, Tham số (orderId).
- **I-Prefix**: Giao diện Interface (IProductRepository).

Kỷ luật Mã nguồn (DRY)

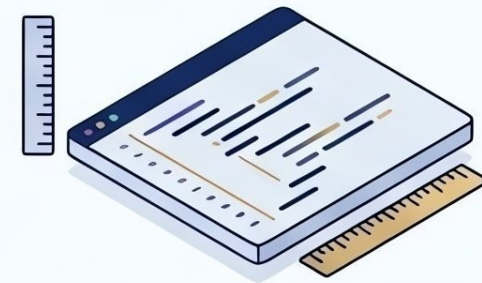
- Hàm đơn nhiệm, ngắn gọn.



- Don't Repeat Yourself: Tuyệt đối không sao chép lặp lại mã nguồn.



- Định dạng thật lề chuẩn xác.



Đạo đức Lập trình & Ranh giới Sử dụng AI



Gia sư, Không phải Thợ viết hộ

- **Nên:** Dùng AI giải thích ✓
lỗi, gợi ý kiến trúc, tối ưu
thuật toán.
- **Không nên:** Sao chép ✗
mà quáng mã nguồn
không hiểu cơ chế.



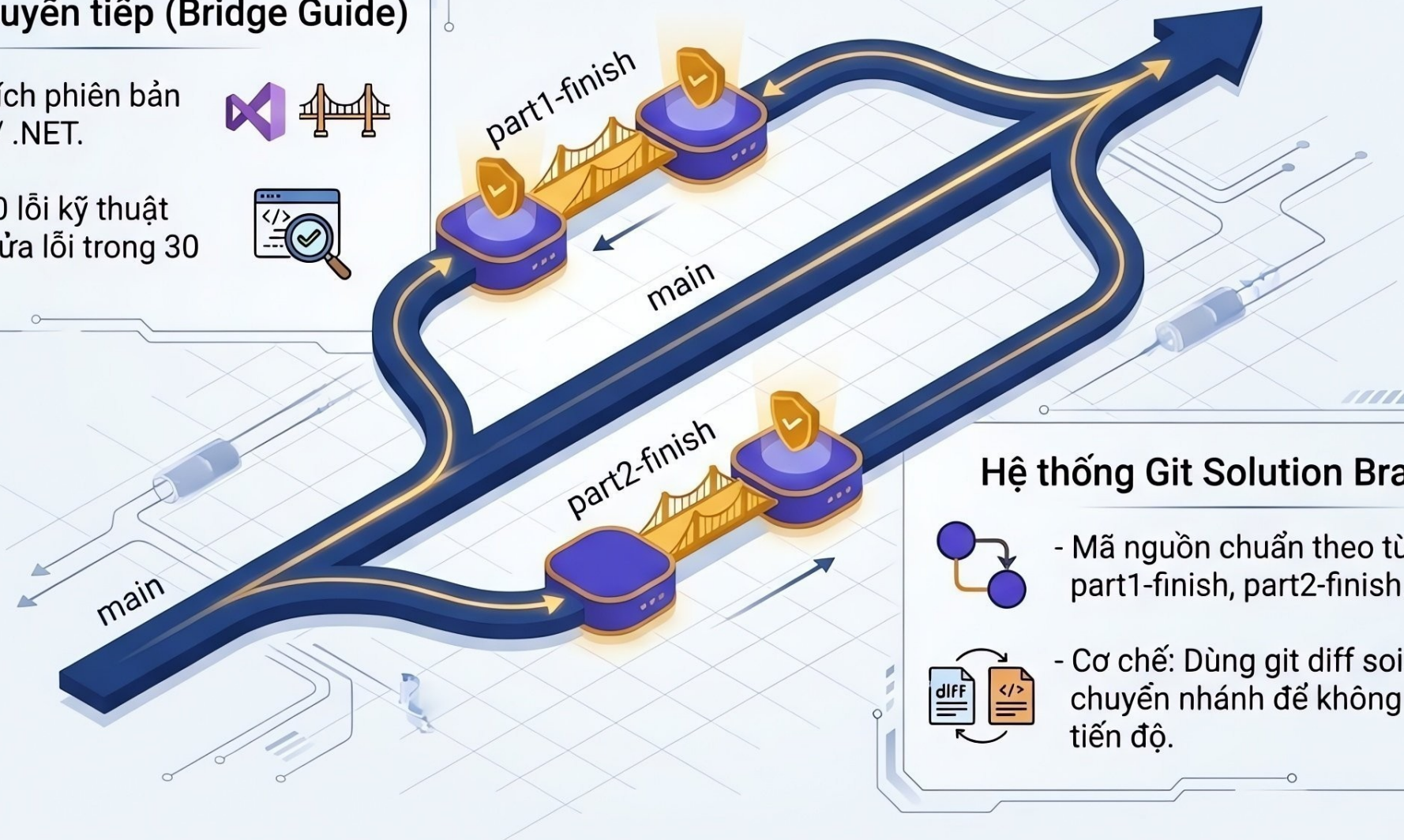
Bảo mật Dữ liệu & Liêm chính

- ✓ - Chịu trách nhiệm
100% từng dòng lệnh
trong dự án.
- ⚠ - Tuyệt đối không đẩy
khóa bảo mật (API
Keys, Passwords) lên
AI công cộng.

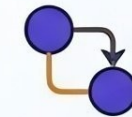
Bộ Công cụ Cứu sinh: Bridge Guide & Git Branches

Cẩm nang Chuyển tiếp (Bridge Guide)

- Xử lý tương thích phiên bản Visual Studio / .NET.
- Tra cứu Top 10 lỗi kỹ thuật thường gặp (Sửa lỗi trong 30 giây).



Hệ thống Git Solution Branches



- Mã nguồn chuẩn theo từng chặng: part1-finish, part2-finish.



- Cơ chế: Dùng git diff soi lỗi hoặc chuyển nhánh để không gián đoạn tiến độ.

Quy trình Tự học: Xem, Hiểu, Tự Code

Bước 3: Tự Code (Code & Verify)

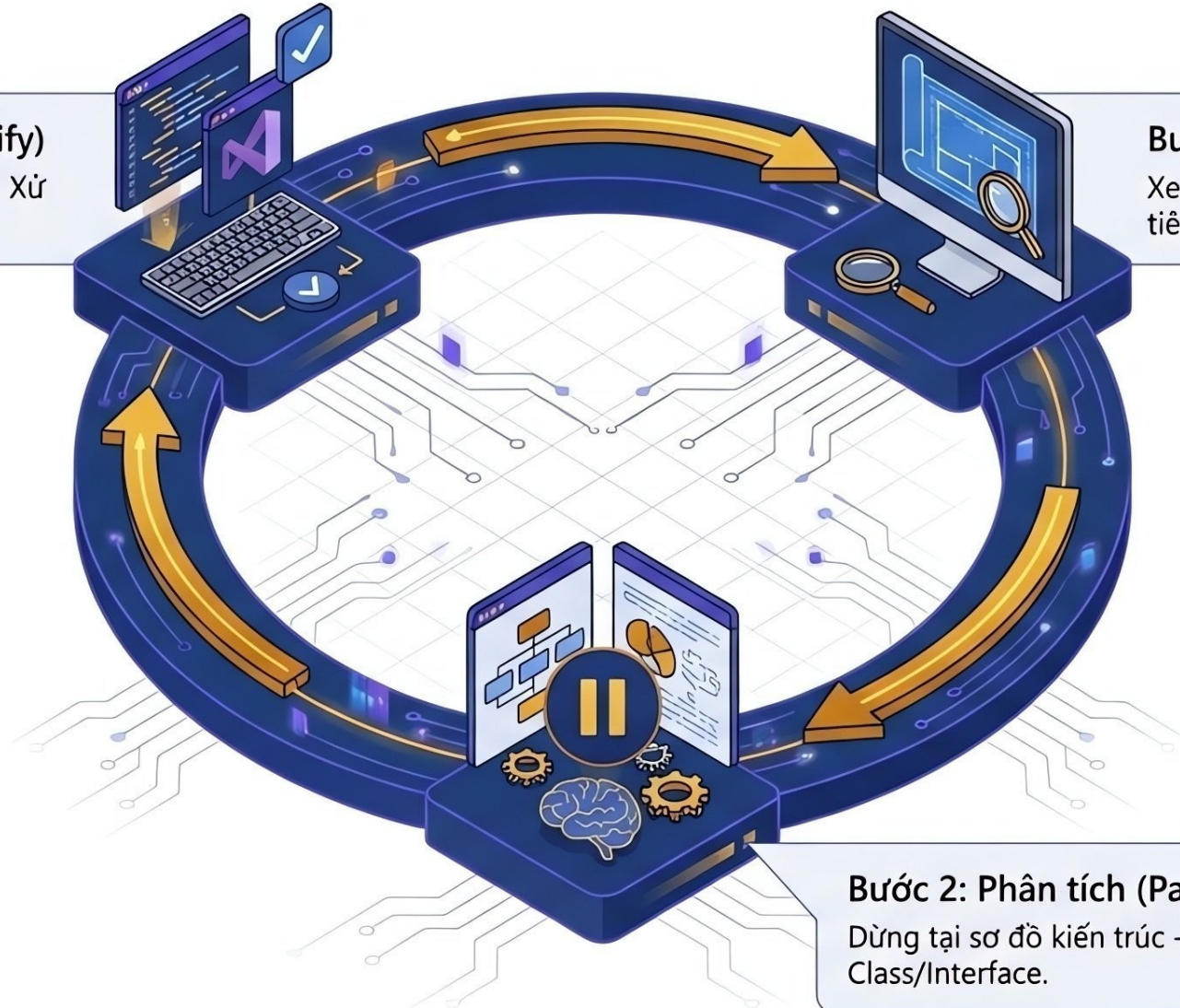
Gõ mã nguồn vào Visual Studio -> Xử lý lỗi -> Kiểm chứng tính năng.

Bước 1: Xem Toàn cảnh (Watch)

Xem lướt tốc độ thường -> Nắm mục tiêu nghiệp vụ và bức tranh lớn.

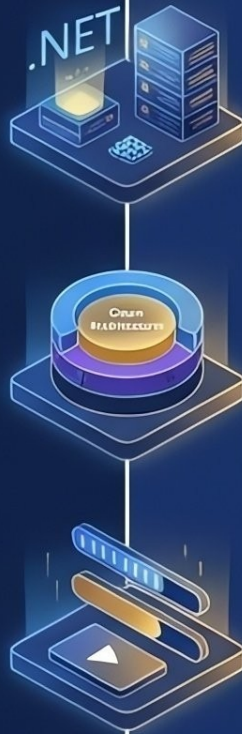
Bước 2: Phân tích (Pause & Understand)

Dừng tại sơ đồ kiến trúc -> Lý giải vì sao thiết kế Class/Interface.



Tóm tắt Part 1 & Câu hỏi Gợi mở

- **Định vị Công nghệ:** .NET là nền tảng vững chắc cho hệ thống lớn và AI.
- **Triết lý Hành động:** Kiến trúc Sạch (Clean Architecture) là kim chỉ nam.
- **Đích đến:** Làm chủ hệ thống eShop qua 35 video thực hành.



Câu hỏi Gợi mở:
Khi gõ một URL, trình duyệt và máy chủ thực sự giao tiếp qua cơ chế nào?



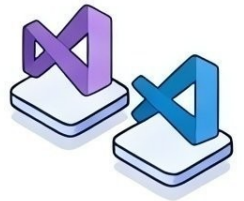
Checklist Khởi động: Chuẩn bị Môi trường



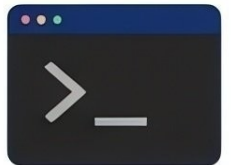
Cài đặt Nền tảng: Tải & Cài đặt .NET SDK bản mới nhất (LTS).



IDE Lập trình: Visual Studio 2022 (ASP.NET workload) hoặc VS Code (kèm C# Dev Kit).



Xác thực Terminal: Chạy lệnh ``dotnet --version`` để xác nhận kết nối SDK.



Tài nguyên: Clone repository khóa học và khởi tạo thư mục thực hành.



Kiến trúc Web & Mô hình Vận hành Blazor

Nền tảng kỹ thuật cho Ứng dụng
Doanh nghiệp C#/.NET

Giao thức HTTP

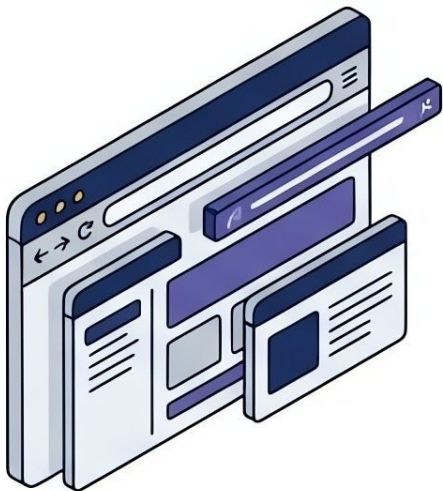
Kết nối SignalR

Cấu trúc Dự án C#



Mục tiêu: Bước đệm kỹ thuật bắt buộc trước khi thực hành chuỗi 10 video eShop (Part 1).

Phía Khách (Browser)



 **Gửi HTTP Request**
(URL, Headers,
Method, Payload)

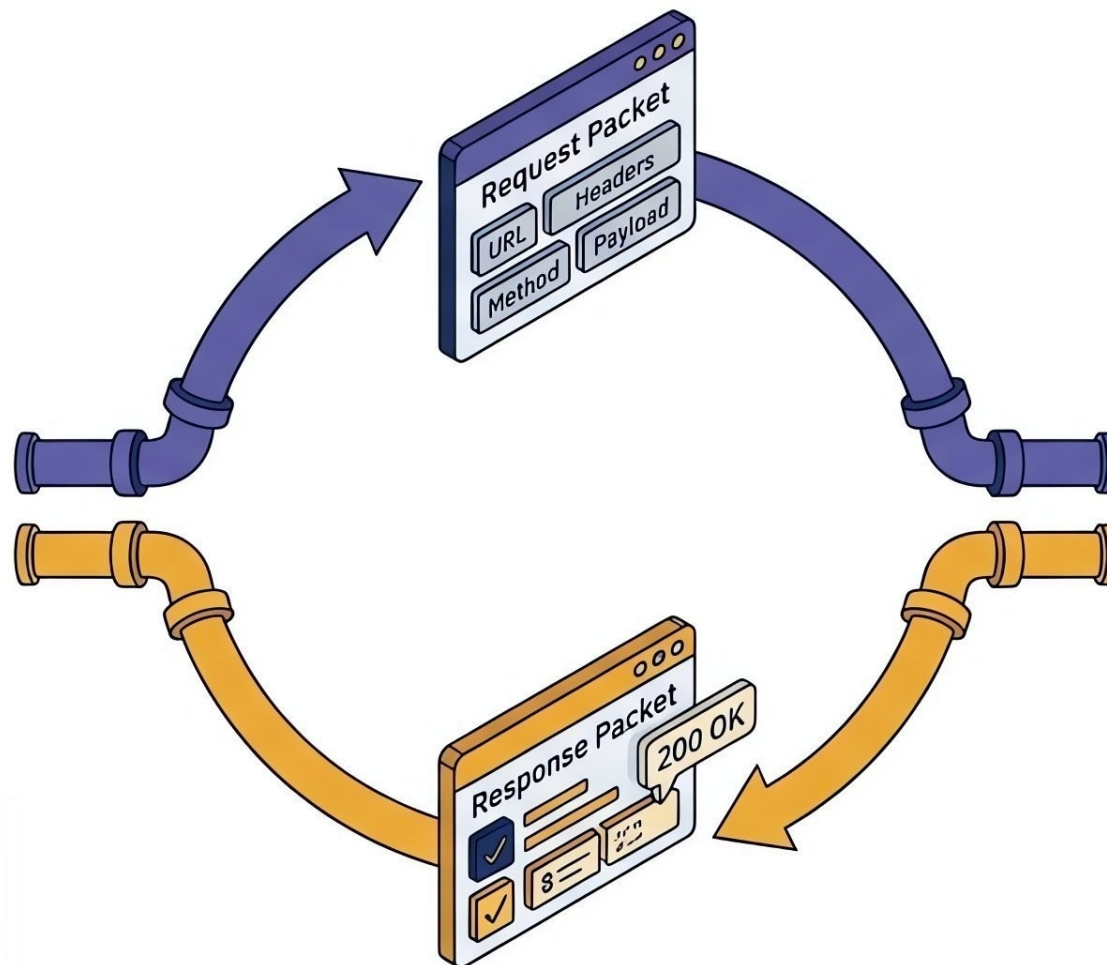
 **Render HTML/CSS/JS**

Phía Chủ (ASP.NET Core)



 **Tiếp nhận yêu cầu, xử
lý nghiệp vụ, truy vấn
CSDL**

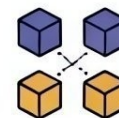
 **Trả HTTP Response**
(Status Code, Data)

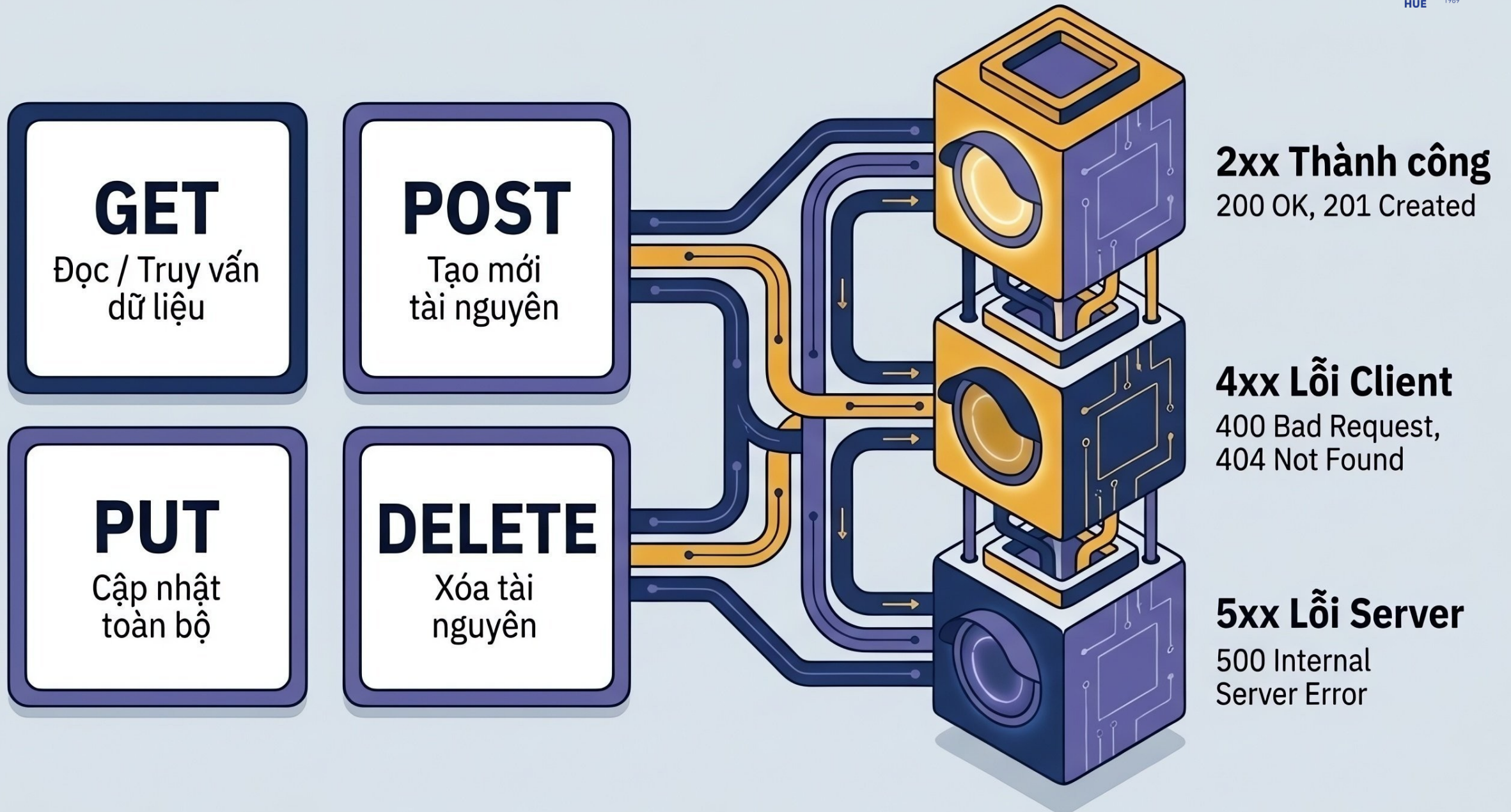


Đặc tính Cốt lõi:

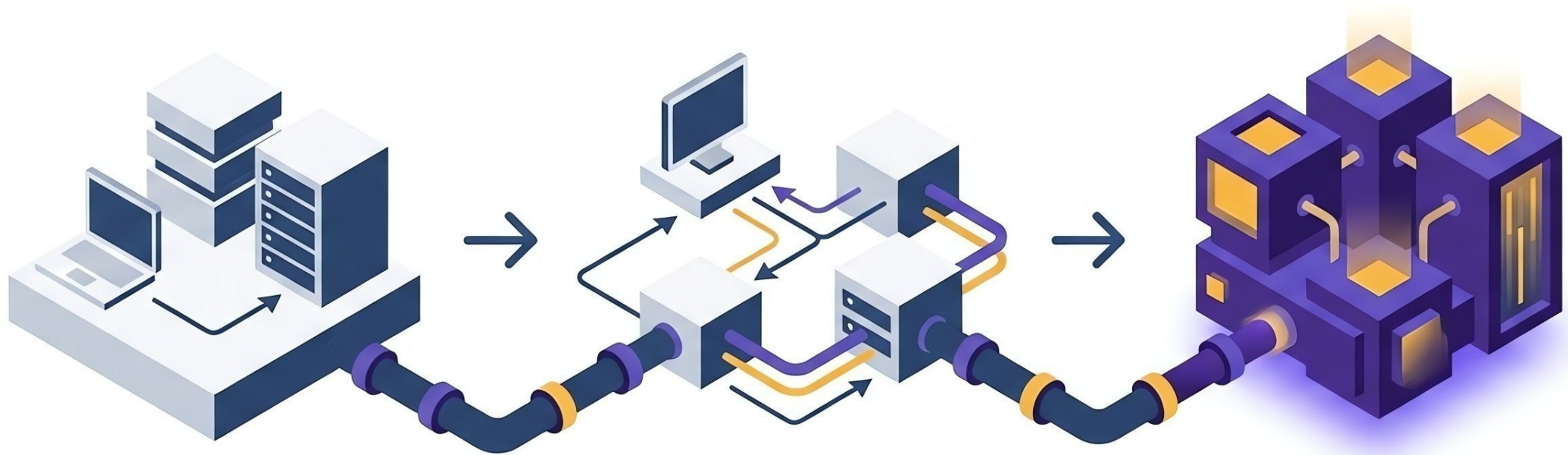


Phi trạng thái (Stateless) –
Mỗi yêu cầu là một **giao dịch**
độc lập, không tự lưu ngữ cảnh.





Lịch sử Tiến hóa Kiến trúc Web



Giai đoạn 1: Static / Server-Rendered

- HTML thuần / ASP.NET MVC
- Click là tải lại toàn trang

Giai đoạn 2: SPA

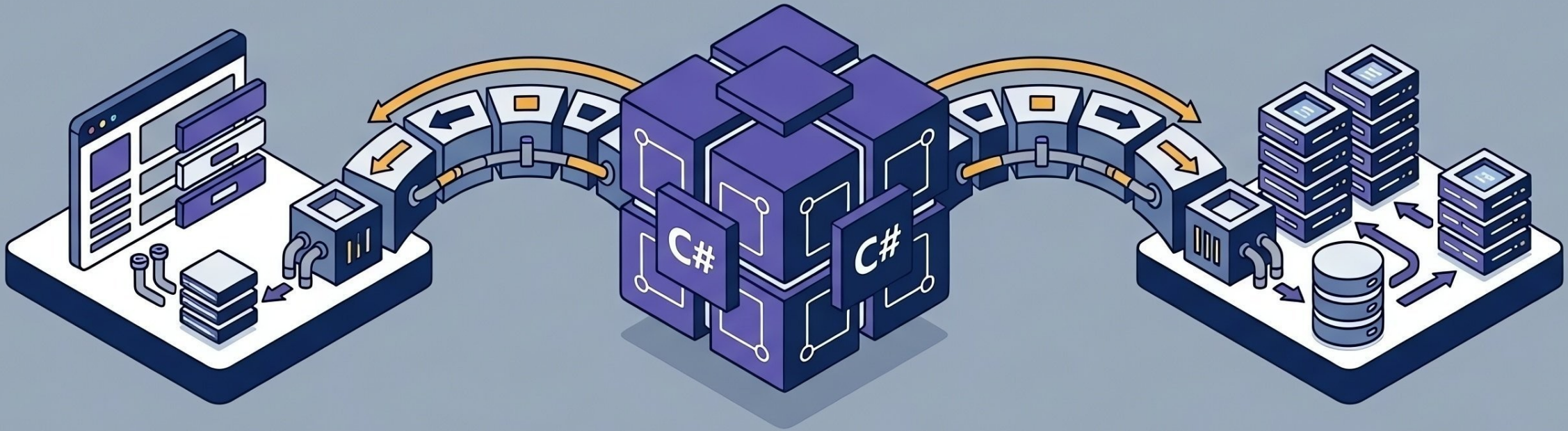
- React / Angular / Vue
- Mượt mà, nhưng phân mảnh 2 ngôn ngữ (JS + C#)



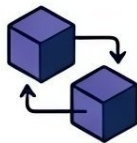
Giai đoạn 3: Fullstack Component

- Blazor
- Viết C# đồng nhất cho toàn bộ hệ thống UI & Backend

Định nghĩa: Framework xây dựng **Interactive Web UI** hoàn toàn bằng **C#**.



1. Tạm biệt hàng ngàn dòng JavaScript rườm rà.



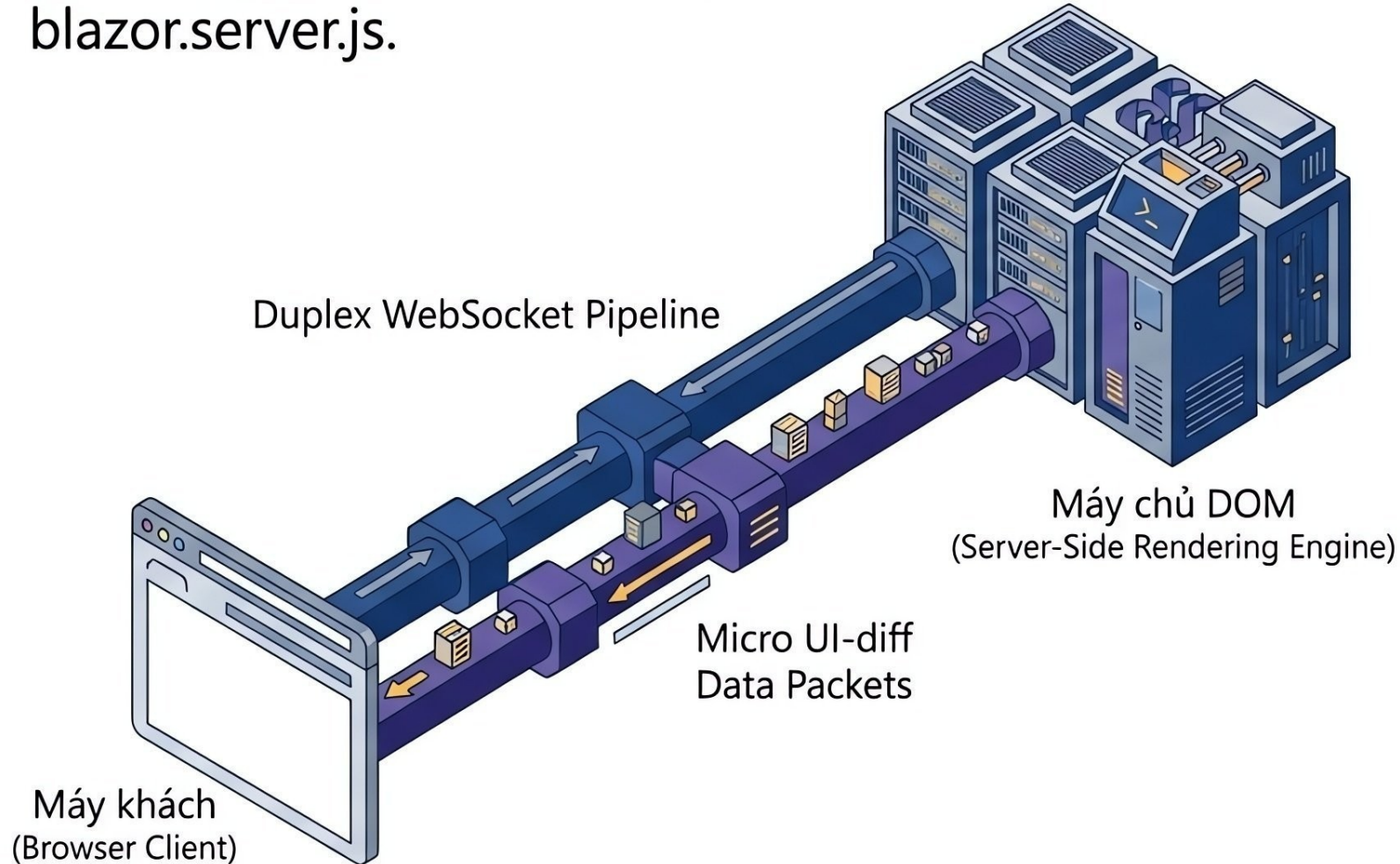
2. Dùng chung Data Models (DTO, Entities) giữa Giao diện & CSDL.



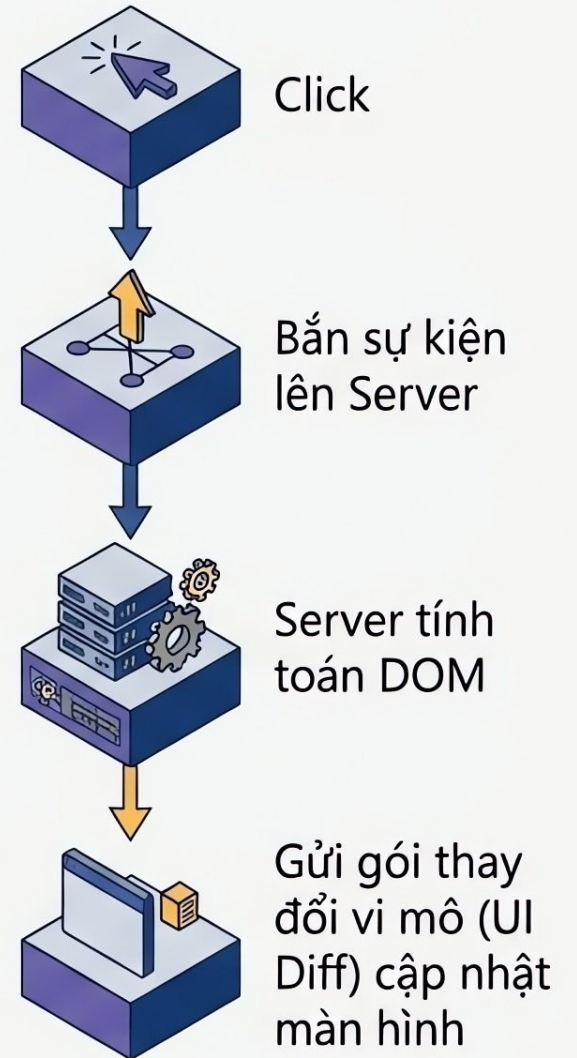
3. Tận dụng trọn vẹn hệ sinh thái .NET & Visual Studio.

Mô hình Thực thi:

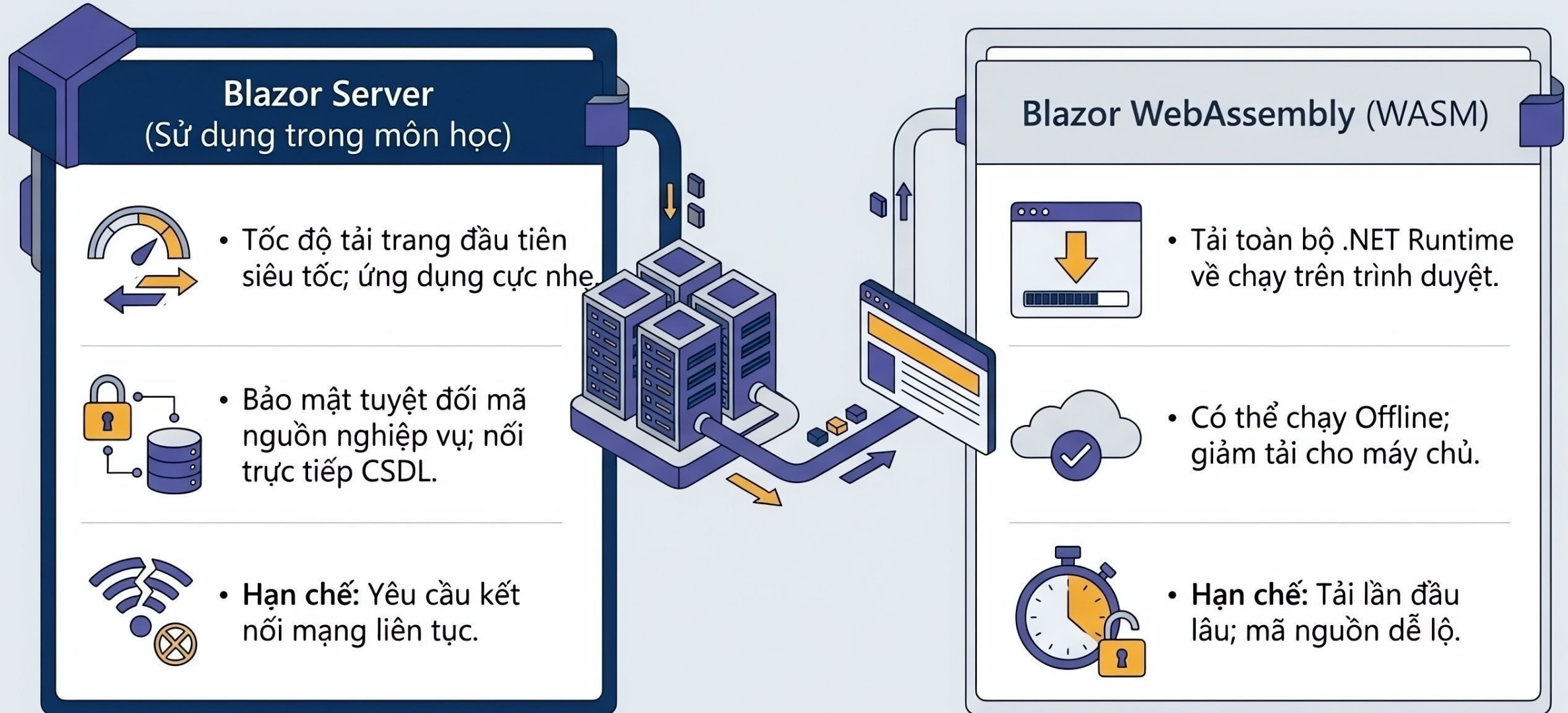
- Mã C# chạy 100% trên Máy chủ.
- Trình duyệt chỉ tải file mỏng blazor.server.js.



Đường ống SignalR (WebSocket)



Bảng So sánh Kiến trúc Blazor



Cấu trúc Thư mục Dự án Blazor Mẫu

Program.cs

Điểm khởi đầu của ứng dụng,
cấu hình dịch vụ và middleware

App.razor

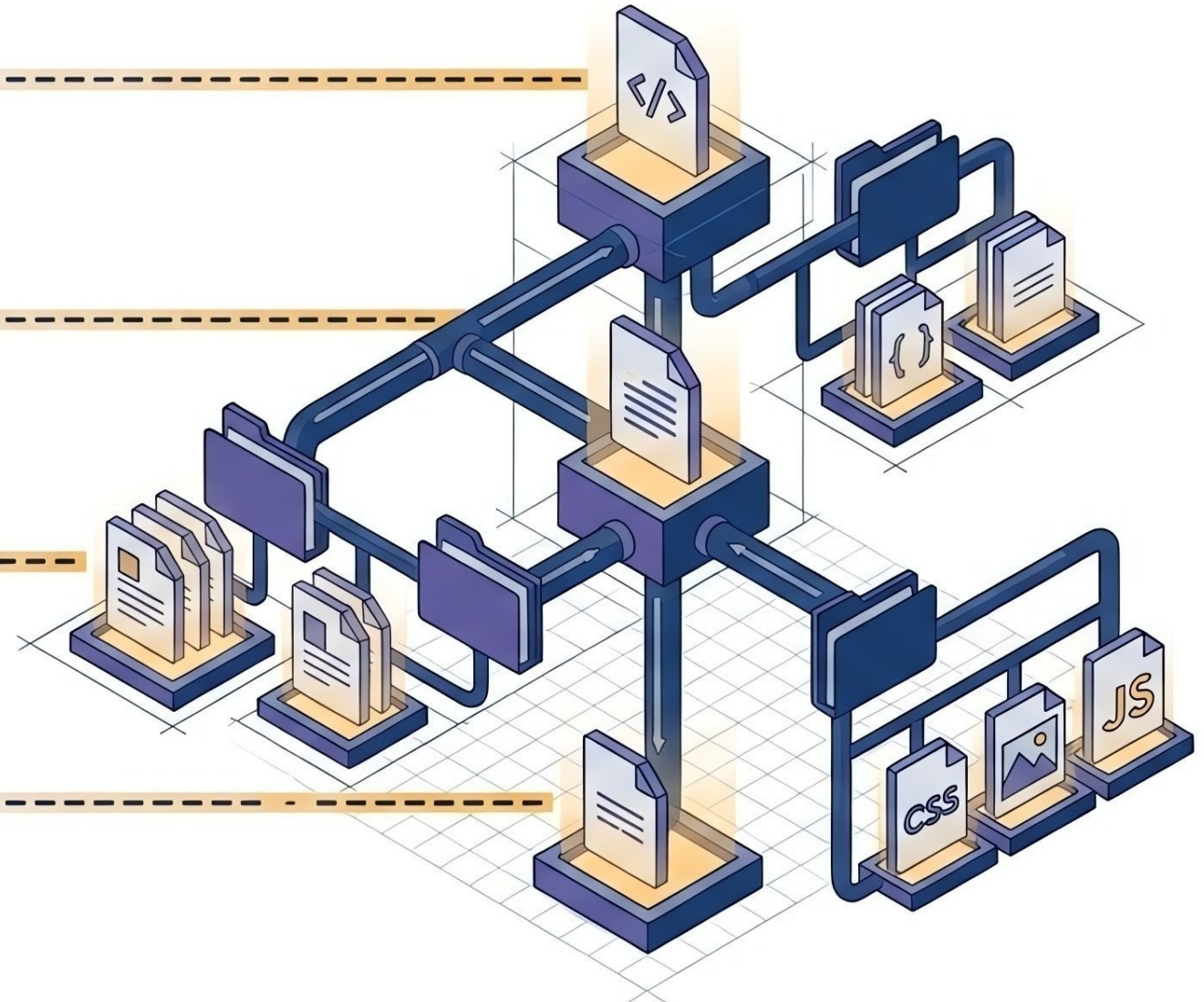
Thành phần gốc điều phối định
tuyến (Router component)

Thư mục Pages/ & Shared/

Chứa các trang Routable Pages
và Layouts dùng chung

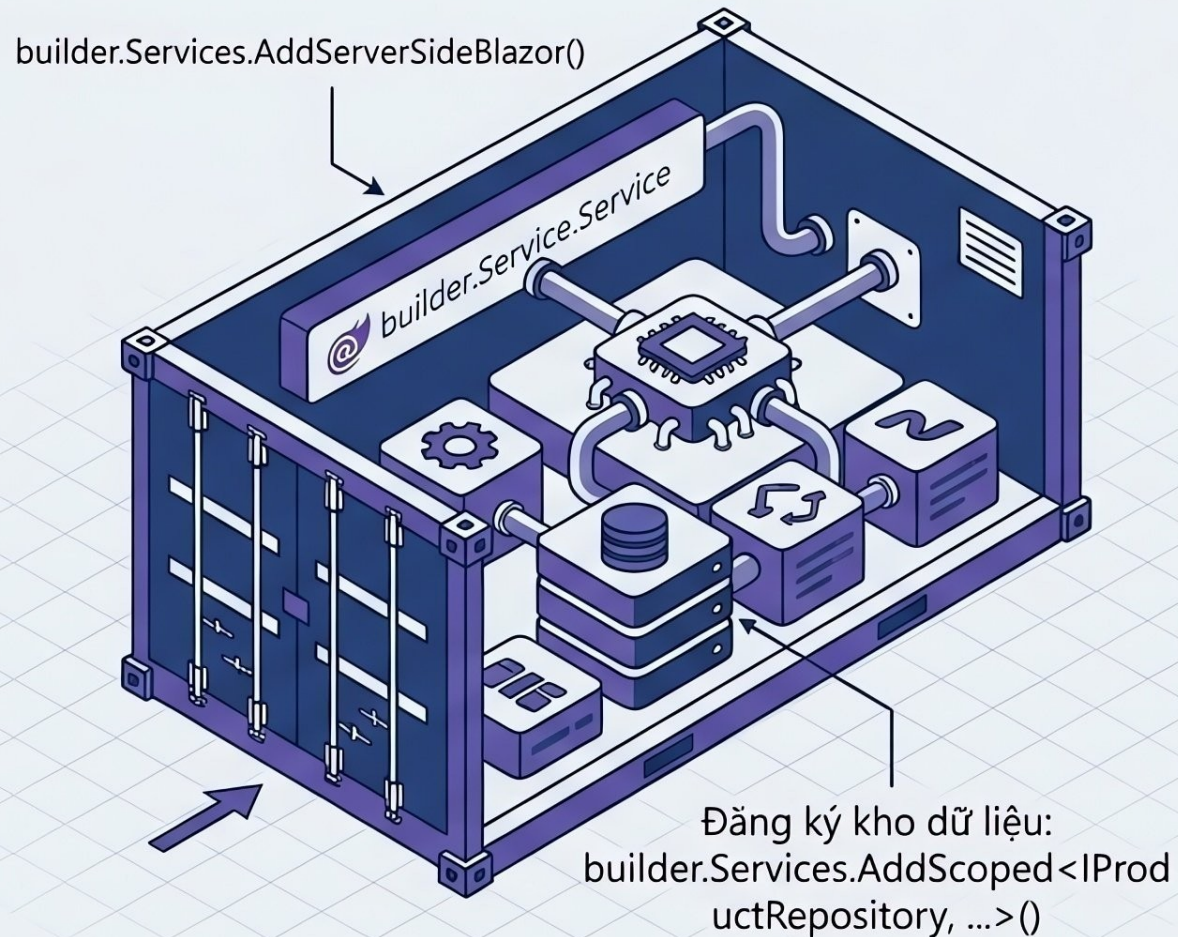
Thư mục wwwroot/

Chứa tài nguyên tĩnh (file CSS,
hình ảnh, JavaScript)

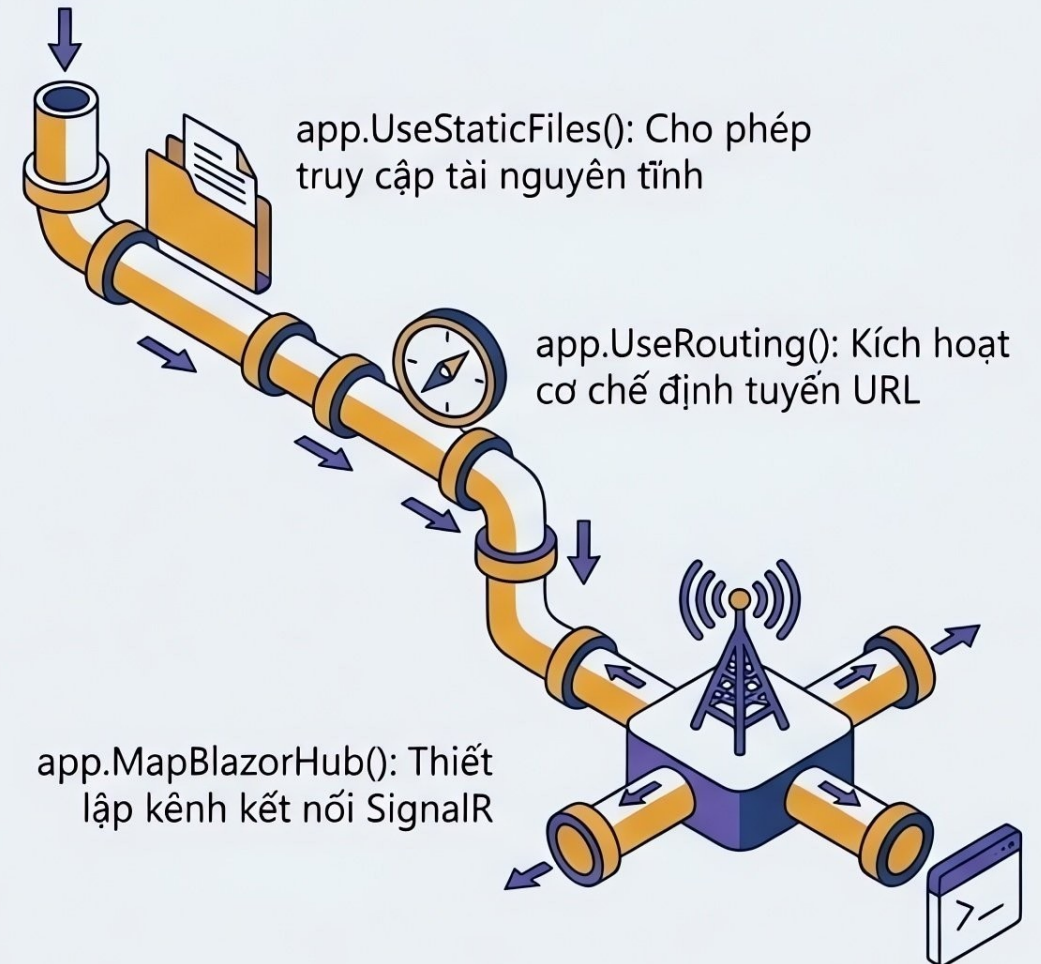


Cấu trúc Nội bộ Program.cs

Khu vực 1: Đăng ký Dịch vụ (DI Container)

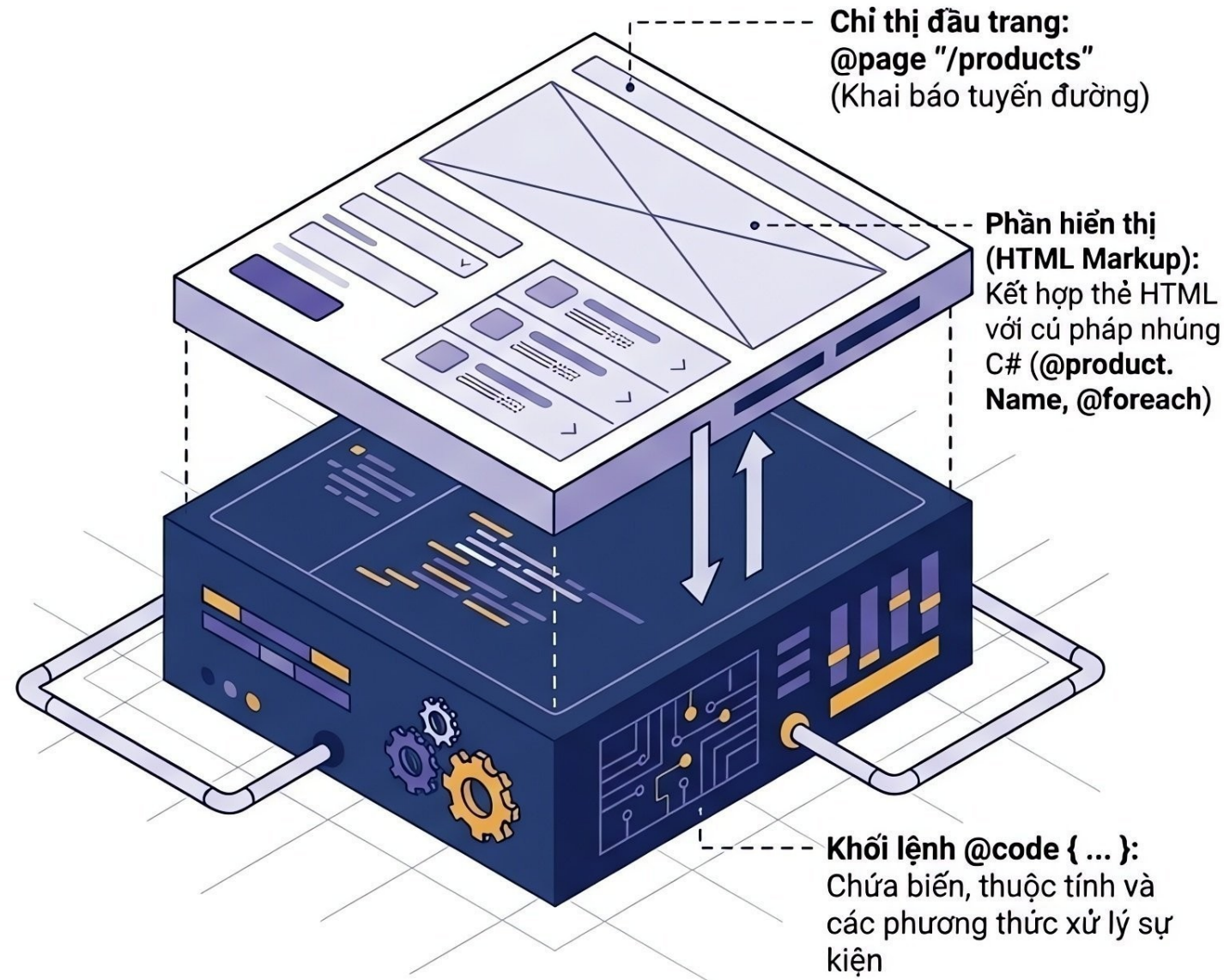


Khu vực 2: Xây dựng Đường ống (HTTP Pipeline)



Ngôn ngữ Razor: Kết hợp HTML Markup & Khối lệnh C# (@code)

Tính Đồng nhất: Không cần chuyển đổi JSON phức tạp, C# tự động bind dữ liệu lên màn hình (Type Safety).

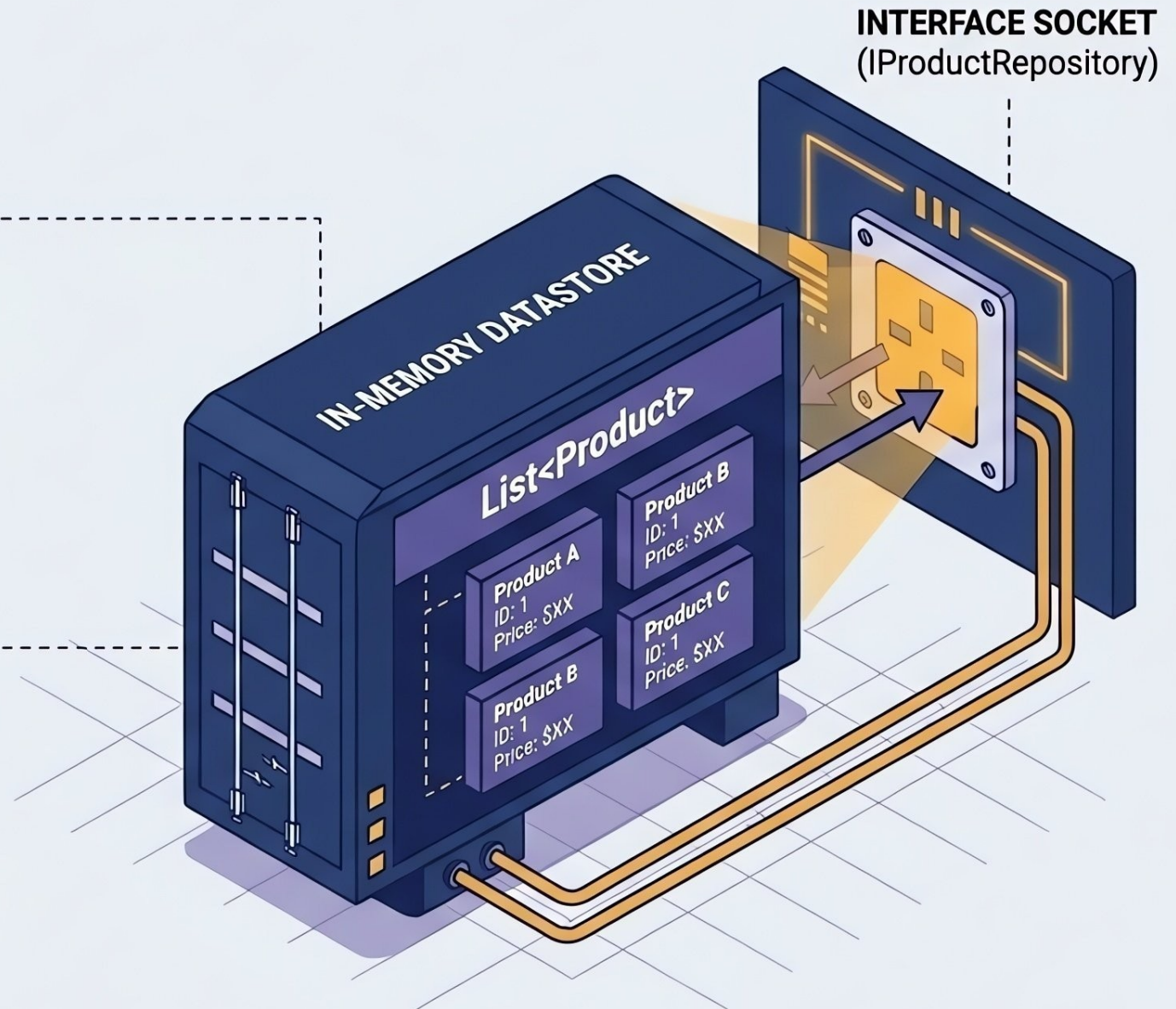


Mục đích Thiết kế (In-Memory DataStore):

Khởi tạo danh sách đối tượng mẫu (List<Product>) chạy trực tiếp trong bộ nhớ RAM. Giúp sinh viên tập trung 100% vào việc làm chủ giao diện và luồng dữ liệu trước.

Đóng gói qua Interface (Abstraction):

Tạo IProductRepository với các hàm GetProducts(), GetProductById(id).
Sau này chỉ cần thay thế bằng SQL
Sau này chỉ cần thay thế bằng SQL Plugin mà không sửa 1 dòng code giao diện!

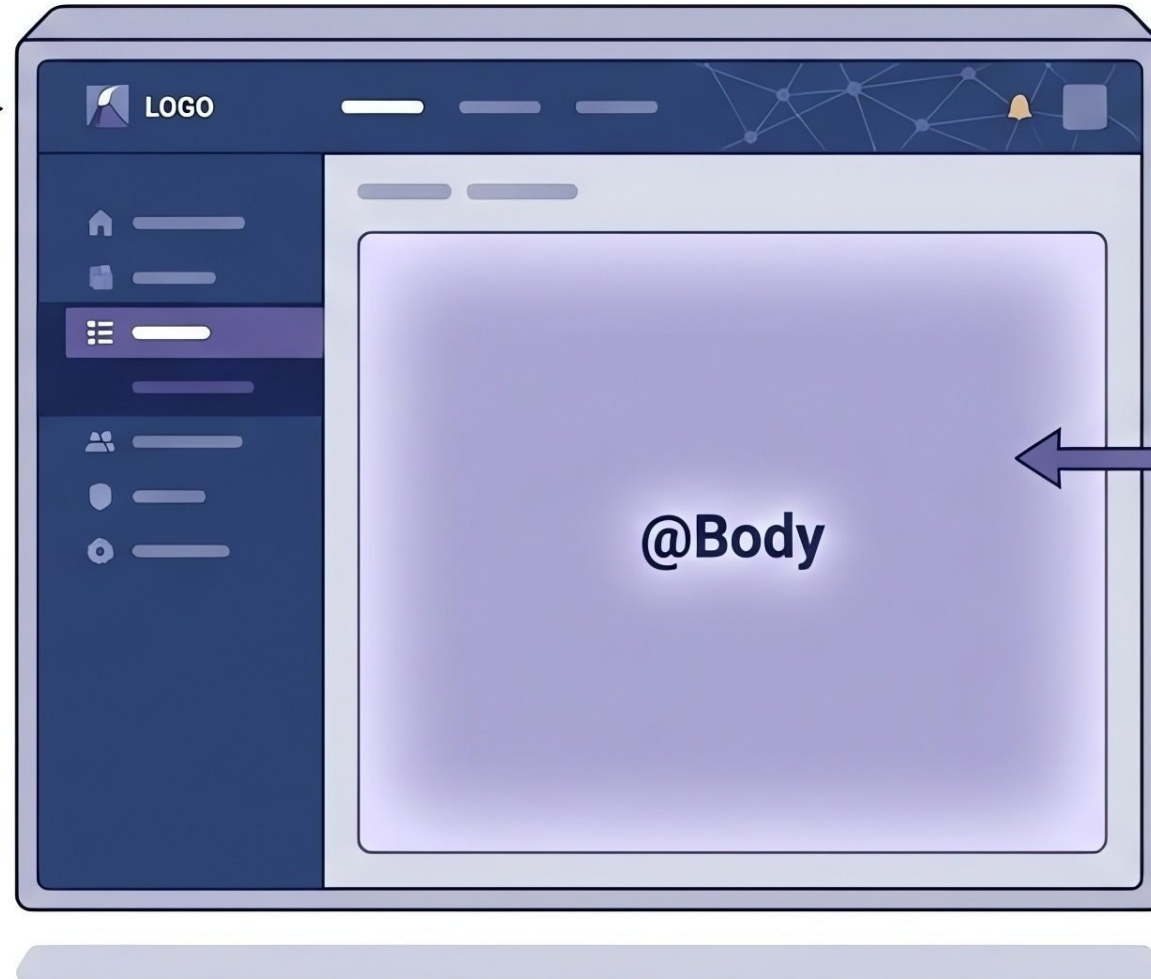


Tổ chức Layout Giao diện: MainLayout & Menu

MainLayout.razor

(Khung Bố cục Toàn cục):

- Chứa thanh điều hướng trên (Top Row), thanh Menu bên cạnh (Sidebar).
- Khoảng **@Body** tự động thay thế bằng nội dung của trang tương ứng khi chuyển URL.

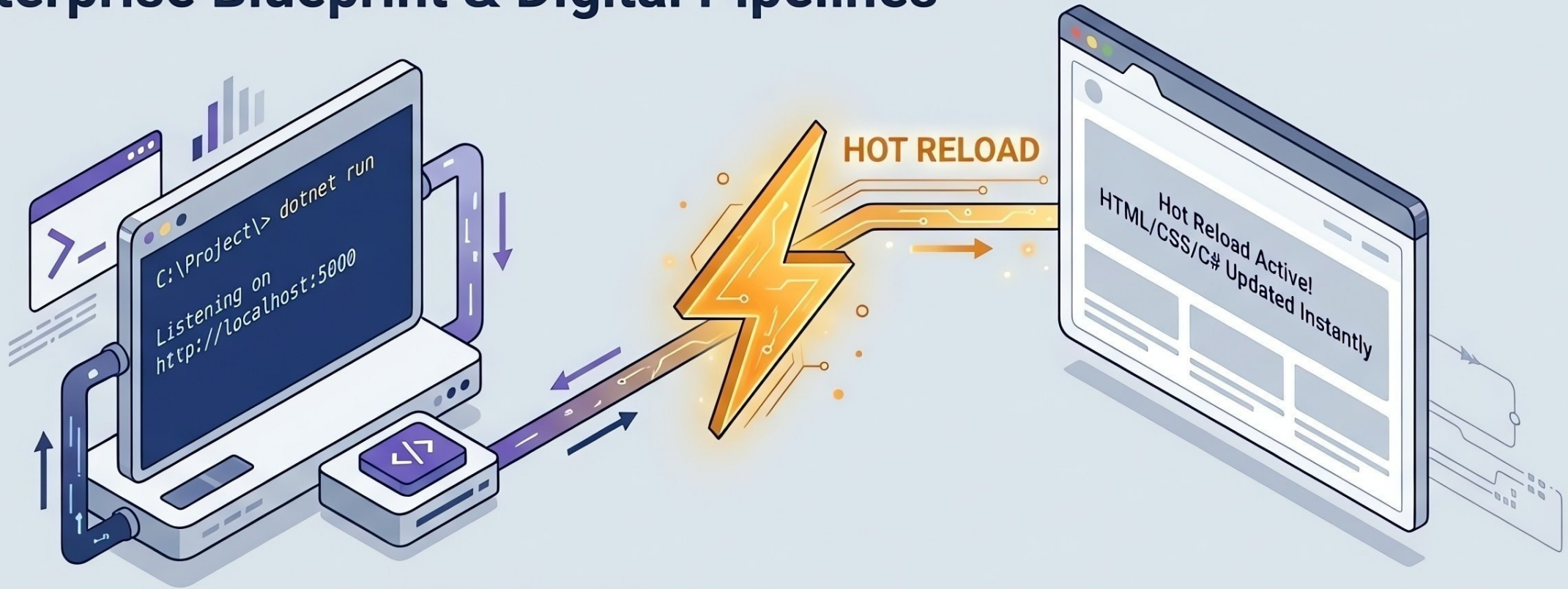


NavMenu.razor

(Thành phần Điều hướng):

- Sử dụng thẻ `<NavLink>` tự động quản lý trạng thái active khi người dùng bấm chọn.

Enterprise Blueprint & Digital Pipelines



Quy trình Khởi chạy qua CLI:

- Mở Terminal tại thư mục dự án -> Gõ lệnh `dotnet run`.
- Ứng dụng lắng nghe tại cổng cục bộ (Localhost).

Trải nghiệm Hot Reload:

- Tự động cập nhật thay đổi HTML/CSS và C# lên trình duyệt ngay khi lưu file (Ctrl+S).
- Phản hồi tức thì, không cần khởi động lại server.

Synthesis Dashboard

Nền tảng vững chắc

- Hiểu rõ chu trình HTTP Request/Response và kết nối thời gian thực SignalR.

Tư duy Mock Data

- Sử dụng In-Memory DataStore qua Interface để tách rời giao diện khỏi CSDL.



Cấu trúc chuẩn mực

- Nắm trọn vẹn vai trò của Program.cs, App.razor và ngôn ngữ Razor.

Sẵn sàng thực chiến!

- Đầy đủ công cụ và tư duy để bắt tay vào dự án eShop.

Nhiệm vụ Tự học: Xem Video Part 1 (10 Videos eShop Cơ bản)

Lộ trình Thực hành eShop:

- **[01-03]:** Khởi tạo dự án, thiết kế Layout trang chủ và In-Memory DataStore.
- **[04-07]:** Xây dựng SearchProductComponent, tách SearchBar và ProductItem.
- **[08-10]:** Tích hợp giao diện Bootstrap 5 và hoàn thiện trang ViewProductDetails.

Yêu cầu Bắt buộc:

- Tự tay tạo dự án, gõ code theo từng video, kiểm thử tính năng.
- Hoàn thành bài tập Checkpoint Part 1 & sẵn sàng cho Bài 2 (OOP & UML).

